VeriReview

Evidence-based verification of GitHub pull-request review resolution

A complete, working, end-to-end system: what it does, how it works, how to run every part of it, its measured results, and what comes next

Status	**Complete and working end to end.** All planned phases (0–13, plus 10.2 and 10.3) shipped; run live as a GitHub App; one optional extension (8b) waits on future data
Headline result	Latest blind test v3 (148 cases): accuracy 0.57, false acceptance 0.09. Across three blind tests false acceptance fell 0.24 → 0.15 → 0.09
Mode	Advisory only: neutral GitHub check; merge blocking built but locked
Author	Nandhagopal
Stack	Python 3.13 · FastAPI · PostgreSQL 16 · Tree-sitter · Docker · GitHub App
Tests	1,266 unit + 39 integration + 2 model tests, strict mypy, ruff, CI

Executive summary

VeriReview is a complete, working system, built end to end and running today. Every component described in this brief exists in the repository, passes its tests, and can be run with the commands in Section 7 — there is no missing piece, stub, or placeholder in the core pipeline.

What was asked	What was built
Detect when a GitHub review comment was not actually addressed	A 7-stage verification pipeline (ingestion → requirements → evidence → rules → aggregation → explanation → policy), fully implemented and tested (Section 2)
Prove it actually works, not just on paper	Three frozen, pre-registered blind evaluations, each run once: false acceptance 0.24 → 0.15 → 0.09; latest accuracy 0.57 on 148 cases with human-labelled real-world cases (Section 6)
Connect it to real GitHub pull requests	A GitHub App: signed webhooks, a durable job queue, a background worker, and a neutral status check on every PR. Run live on a test repository on 8 October 2026 (Section 4.2)
Make it safe by default	Merge-blocking is fully built but deliberately locked behind four independent switches, all closed today (Section 4.3); repository content is always treated as untrusted data (prompt-injection tested: 0 / 1,288)
Give an operator visibility	A read-only, token-protected web dashboard with no JavaScript (Section 4.4), seedable with a one-command demo dataset so it can be seen working immediately (Section 7.3)
Be reproducible and maintained	1,266 unit + 39 integration tests, strict typing, linting, and CI on every push (Section 5)

What is genuinely still open is data, not software. Human labels now exist: one annotator labelled every real-world test case, and a second annotator labelled 36 cases with the disagreements adjudicated. But on hard cases even two humans agreed only 5 times in 21, and the latest round of rule work barely transferred to new cases (2 of 148 verdicts changed). The next step is therefore more, and more reliably labelled, real-world data, before any further rule change or any thought of merge-blocking (Sections 8 and 9).

Contents

Executive summary

1. The problem and the one question VeriReview answers

2. How it works: the verification pipeline

3. A worked example (real output)

4. Deployment pieces: API, GitHub App, enforcement gate, dashboard

5. Technology stack and project layout

6. Current stage: phases, results, honest limitations

7. How to run the whole system (step by step)

8. Future plans

9. Human annotation: what was done and what remains

10. Review preparation: likely questions and answers

11. Glossary

How to read the numbers. Every number in this brief comes from the project's recorded evaluation reports (`experiments/*.json`) and README. "Blind" means the test cases were frozen by hash before the rules were changed and evaluated exactly once.

1. The problem

On GitHub, a reviewer leaves a comment on a pull request ("validate `username` and return HTTP 400"). The author pushes changes and clicks **Resolve conversation**. Nothing checks that the change actually did what was asked: a thread can be resolved with a partial fix, a fix in the wrong place, or only a comment that mentions the fix.

VeriReview answers one question:

Did the code changes made after the review comment actually satisfy the review requirement?

Its answer is a verdict backed by cited evidence (file, lines, commit):

Verdict	Meaning
SATISFIED	every requirement in the comment is met by the code at the end of the window
PARTIALLY_SATISFIED	some requirements met, some not
NOT_SATISFIED	no requirement met (nothing changed, wrong place, only a comment, wrong value…)
UNCERTAIN	the request is ambiguous, or the rules cannot verify it: goes to a human

A separate **policy** layer turns verdict + confidence into an action: ALLOW / WARN / HUMAN_REVIEW / BLOCK. **BLOCK is disabled**: the system is advisory until it is statistically proven safe per category (Section 4.3).

Design principles

- **Evidence, not opinion.** Every conclusion cites the structural fact it rests on.
- **Deterministic rules decide; models only advise.** A semantic model is one evidence source, never the final authority (Phases 6–7 showed similarity models accept bad fixes).
- **When a rule cannot verify, say UNCERTAIN, never accept.** False acceptance (a bad fix accepted) is the safety-critical metric.
- **Repository content is untrusted data.** Code comments, commit messages and replies are never instructions (prompt-injection tested). Repository code is never executed.
- **Honest evaluation.** Test sets are frozen by hash before rule changes and used once.

2. How it works: the verification pipeline

```
GitHub PR → ingestion → ReviewCase → requirement extraction → evidence → aggregation → policy
 (REST +     thread +     before/after   comment -> atomic,       diff, AST,   verdict +      ALLOW / WARN /
  GraphQL)   commit window code, diff,   categorised requirements rules, tests confidence +   HUMAN_REVIEW
             + flags       tests         + ambiguity score        code model*  explanation    (BLOCK locked)
```

* the code model runs only in the optional `phase7-semantic` pipeline and is neutral.

Stage	What it does	Phase
Ingestion	Rebuilds the review thread from GitHub (REST + GraphQL for resolution state) and the **temporal window**: the commits made after the comment. Handles rebases / force-pushes and sets reliability flags (e.g. resolution time unknown).	1
ReviewCase	One self-contained record: thread, code before and after, diff, changed tests, flags (Pydantic contracts shared across the system).	2
Diff + syntax	unidiff / difflib diffs; Tree-sitter parses Python into symbols and structural facts (calls, conditions, raises, handlers, returns). Finds the commented function even if it was renamed, moved, or extracted into a helper.	3
Requirements	Rule-based extraction: splits the comment into atomic requirements (one action, one target), assigns a category (naming, validation, testing, error_handling, api_behavior, other), targets, conditions, GitHub suggestion blocks, and an **ambiguity score**.	4
Rules	One deterministic rule per category that queries code structure: guards, handlers, HTTP responses, test inputs. A comment that only mentions the fix never counts. Suggestion blocks are checked against their exact code; common other requests (remove this, add a docstring, use A instead of B) have rules too.	5, 10.1
Aggregation	Per-requirement status → overall verdict. Ambiguous requests → UNCERTAIN. Confidence is lowered when the case is unreliable. HIGH confidence is never emitted.	5, 8a
Explanations	Human-readable explanation that cites each evidence id with file:lines@commit.	8a
Policy	Verdict × confidence → action, in observe / advisory / human_review / enforcement mode. BLOCK additionally needs an eligible category and all four locks open.	8a, 12
NLP baselines	Keyword overlap, TF-IDF and sentence-embedding (MiniLM) similarity, scored by the same harness, for comparison only.	6
Code model	UniXcoder relates each requirement to the most relevant added code (comments removed). **Neutral evidence**: it never changes a verdict (ADR-002).	7

Why rules and not a model?

Phases 6–7 measured similarity models directly. They reward a comment that repeats the reviewer's words, and even genuinely relevant code can be wrong (the right check in the wrong place still scored 0.73). At any usable threshold they accepted 12.5–87.5% of bad fixes on held-out data, versus 0% for the rules. On the latest blind test (148 cases) the code-model and structural baselines accepted 70–98% of bad fixes, versus 9% for the rules. So models are kept as informational evidence only.

3. A worked example (real tool output)

Request: "validate `username` and return HTTP 400". The developer added the validation but no 400 response. Output of `verireview verify-fixture dataset/fixtures/api-002-validation-without-400`, abridged to 6 of its 19 evidence lines (the tool's □ / □ marks are shown as + / x):

```
Requirements:
+ R1 (validation): validate `username` (non-empty, max 32 chars)
x R2 (api_behavior): return HTTP 400 on invalid input

Evidence:
• [E3] Commented line 10 is in function `signup`. (web/signup.py:7-11 @ 64d302f (before))
+ [E15] Emptiness check `not username or len(username) > 32` on `username` raises `ValueError`
+ [E16] Range check `not username or len(username) > 32` on `username` raises `ValueError`
+ [E17] R1 (validation) satisfied: `username` is checked before use.
x [E18] No response with HTTP 400 on the requested branch.
x [E19] R2 (api_behavior) not_satisfied: HTTP 400 is not returned where requested.

Result: PARTIALLY_SATISFIED     Confidence: MEDIUM
Recommended action: HUMAN_REVIEW (advisory mode). Only part of the request appears satisfied.
```

4. Deployment pieces

4.1 HTTP API (FastAPI)

Endpoint	Purpose
POST /verify	verify a ReviewCase sent in the body
POST /verify/github	fetch a thread from GitHub (repo, PR, comment id) and verify it
POST /github/webhook	GitHub App deliveries; HMAC-SHA256 signature checked on the raw body before parsing (401 if forged, 503 if no secret configured)
GET /dashboard…	read-only operator pages (404 unless a dashboard token is configured)
GET /health, /health/db	liveness and database readiness

4.2 GitHub advisory mode (Phase 11)

```
Thread resolved / commit pushed / check re-run -> signed webhook -> POST /github/webhook (HMAC)
   -> job row in PostgreSQL queue (unique delivery id; claimed with SKIP LOCKED; retry + backoff)
   -> worker: App JWT -> installation token scoped to ONE repo (contents/PR read, checks write)
   -> re-reads thread + code from GitHub -> pipeline -> audit row (inputs sha256, full result)
   -> one neutral "VeriReview" Check Run on the PR head (with a "Confirm reviewed" button)
```

- Redelivered webhooks are processed once (idempotent queue).
- The webhook body is never trusted: only validated ids are taken; everything is re-read from GitHub.
- Secrets (App key, webhook secret) come from settings as `SecretStr`, never logged.
- **Run live on 8 October 2026** on a private test repository. The live run found 7 problems that recorded tests had missed (no worker logs, a wrong cited line, the Re-run button's event type, unlogged events, a stale check after a push, one error stopping the worker, tests sharing the live database); all were fixed and covered by tests.

4.3 Staged enforcement (Phase 12): built but inert

A check can only turn into a failing (blocking) check when **four independent locks** are all open:

#	Lock	Today
1	Global switch `VERIREVIEW_POLICY_ALLOW_BLOCK`	off (pinned by a test)
2	Per-repository stage: observe → advisory → human_review → enforcement, one step at a time, recorded; enforcement only after ≥ 14 days of human review and ≥ 10 reviewer confirmations	no repository promoted
3	Eligible category: one-sided 95% Clopper–Pearson upper bounds of false acceptance and false blocking both ≤ 5% on a frozen test run (ADR-003)	**none eligible**
4	The repository itself marks the check as required (branch protection)	repository's choice

Why no category is eligible: with 0 errors in n cases, the 95% upper bound is about 3/n, so you need at least **59 clean cases** of each kind per category to get under 5%. On the latest (v3) test each category has 10–12 bad and 7–14 good cases, and the upper bounds of false acceptance are 22–51%.

4.4 Read-only dashboard (Phase 13)

- Server-rendered in FastAPI with Jinja2 templates, **no JavaScript**, strict Content-Security-Policy, `Cache-Control: no-store`.
- Off by default: enabled only when `VERIREVIEW_DASHBOARD_TOKEN` is set. Sign-in with that token creates an HMAC session cookie (HttpOnly, SameSite=Strict, 8 h); failed sign-ins are rate-limited (10 per 5 min).
- Pages: overview (verdict and job totals, gate status), repository (stage, confirmations, history), pull request (threads and verdicts), audit detail (thread, requirements, evidence, explanation, code before/after, diff, inputs hash), jobs queue.

- Stored code is private: `verireview purge-audit` removes it after 90 days (verdict, hashes and evidence stay as the audit trail).

5. Technology stack and project layout

Layer	Technology
Language / tooling	Python 3.13, uv (dependency lock), ruff (lint + format), strict mypy, pytest
Parsing	Tree-sitter (Python grammar, pinned), unidiff / difflib
Service	FastAPI + uvicorn, Pydantic contracts, Jinja2 (dashboard)
Storage	PostgreSQL 16 (Docker, host port 5433), SQLAlchemy, Alembic migrations 0001–0005
GitHub	REST + GraphQL client; GitHub App auth with PyJWT[crypto] (RS256)
Optional NLP group	scikit-learn, sentence-transformers (MiniLM), transformers (UniXcoder), PyTorch CPU; never in the service image (526 MB)
Deployment	Docker Compose: db, api, worker (profile `advisory`); GitHub Actions CI

```
src/verireview/
  gh/ threads/ ingestion/   GitHub client, thread reconstruction, temporal window, ReviewCase
  contracts/                shared Pydantic models (ReviewCase, Requirement, Evidence, Result)
  diff/ syntax/             diffs; Tree-sitter symbols, facts, location resolution
  requirements/ rules/      requirement extraction (lexicon in one file); one rule per category
  evidence/ verification/   evidence stages, pipelines, aggregation, ambiguity gate, reliability
  explanations/ policy/     cited explanations; ALLOW / WARN / HUMAN_REVIEW / BLOCK
  semantic/ evaluation/     baselines, UniXcoder, metrics, injection suite
  benchmark/                splits, mining, pseudonymisation, annotation page, kappa, gold, freeze
  advisory/ enforcement/    GitHub App (webhooks, queue, worker, check); stages + eligibility gate
  dashboard/ api/ db/       read-only pages; HTTP routes; database models
dataset/  experiments/  docs/ (roadmap, per-phase docs, ADRs)  migrations/  tests/
```

6. Current stage

6.1 Phase progress

Phase	What	Status
0	Skeleton: uv, FastAPI /health, PostgreSQL + Alembic, Docker, CI	Done
1	GitHub ingestion, temporal window, live check on real PRs (8/10 threads)	Done
2	Contracts, 29 dev fixtures, evaluation harness, locality baseline	Done
3	Diff + Tree-sitter symbols, facts, location resolution	Done
4	Requirement extraction + ambiguity; blind held-out: count 0.844, category F1 0.909	Done
5 / 5.1	Per-category rules; held-out blind 0.625 (FAR 0.000), then hardening	Done
8a	MVP: confidence, policy, explanations, API, end-to-end tests (15/15 criteria)	Done
6	NLP baselines (lexical, TF-IDF, MiniLM) through the same harness	Done
7	UniXcoder as neutral evidence; prompt-injection suite (0 changes in 2,576 variants)	Done
9	Benchmark v1: annotation guide, blind test cases, 50 real-world cases, frozen manifest, annotation tooling	Done
10	Pre-registered blind evaluation + ablation (acc 0.48, FAR 0.24)	Done
10.1	Benchmark v2 frozen first; rule fixes on dev; blind v2 run (acc 0.65, FAR 0.15)	Done
10.2	Human labels (one annotator) for v2 and v3 real-world; benchmark v3 frozen before any rule change	Done
10.3	Rule fixes on dev; one blind v3 run, old vs new (acc 0.57, FAR 0.09; 2 of 148 changed)	Done
11	GitHub advisory mode; live run on a test repository (7 problems found and fixed)	Done
12	Staged enforcement: four locks, statistical gate; inert (nothing eligible)	Done
13	Read-only dashboard, token off by default, retention purge	Done
—	Demo data: one command seeds the dashboard for a live walkthrough (Section 7)	Done
8b	Optional extension: semantic evidence in the verdict, once data justifies it (ADR-002)	Future
—	Second annotator on 36 v2 cases (κ 0.33), 19 disagreements adjudicated	Done
—	More two-annotator data and benchmark v4 before further rule work (Section 9)	Future

All core phases (0–13, plus 10.2 and 10.3) are complete, tested and running. The two "Future" rows are not unfinished parts of the shipped system — they are optional next steps the roadmap always scoped as separate, data-gated work (Section 8).

6.2 Headline results: latest blind test v3 (148 cases)

60 controlled + 30 adversarial + 58 fresh real-world cases (labelled by one human annotator), frozen **before** any rule change and evaluated once with a pre-registered protocol, old rules against new. 95% bootstrap intervals in brackets.

System	Accuracy	Macro-F1	False accept. ↓	False block. ↓	Coverage
A lexical overlap	0.318	0.158	0.018	0.889	1.00
B embeddings (MiniLM)	0.304	0.142	0.000	0.931	1.00
B′ UniXcoder, code view	0.507	0.245	0.825	0.097	1.00
L change near comment	0.480	0.174	0.982	0.014	0.99
S AST structure	0.561	0.302	0.702	0.069	0.99
F-old: VeriReview, Phase 10.1 rules	0.561 [0.48, 0.64]	0.596	0.105 [0.03, 0.19]	0.111	0.58
F: VeriReview, Phase 10.3 rules	**0.568 [0.49, 0.64]**	**0.602**	**0.088 [0.02, 0.17]**	**0.111**	**0.57**
F with gold requirements	0.608	0.635	0.053	0.153	0.62

- **Old vs new rules on the same cases** (paired bootstrap): accuracy +0.007 [0.00, +0.02], false acceptance −0.018 [−0.06, 0.00]. Only 2 of 148 verdicts changed. The pre-registered safety question is met; the improvement question is not.
- **Fixes did not transfer:** on development data the same changes fixed 11 cases (false acceptance 6.1% → 0.7%); the dev–test gap grew from 0.145 to 0.203.
- **Baselines that look accurate accept almost everything** (L, S, B′: 70–98% false acceptance). Baselines with low false acceptance reject almost everything (A, B: 89–93% false blocking).
- **Real-world:** 13 of 58 decided, 8 correctly, no bad fix accepted; the rest go to human review.

6.3 Other results

Measure	Result
Blind test v1 (Phase 10, 120 cases)	accuracy 0.483 [0.39, 0.57], FAR 0.237 [0.14, 0.35]
Blind test v2 (Phase 10.1, 147 cases)	accuracy 0.653 [0.58, 0.73], FAR 0.151 [0.06, 0.26]; old → new rules on the same cases: accuracy +0.231, FAR −0.151
Human labels	one annotator for all v2 and v3 real-world cases (κ 0.60 against the provisional model labels, which were too lenient); second annotator on 36 v2 cases: κ 0.33 (random 15: 15/15 agree; disputed 21: 5/21), 19 adjudicated; re-scored v2 real-world: 0 of 4 bad fixes accepted
Requirement extraction, blind held-out	count exact 0.844, category F1 0.909 (targets 0.80 / 0.85)
Prompt injection (4 texts × 7 sites × every fixture)	0 / 1,288 outcome changes for mvp and for phase7-semantic; the lexical baseline flipped 6 (NOT_SATISFIED → SATISFIED)
Code-model evidence (UniXcoder)	ROC-AUC dev 0.569 / held-out 0.737, neutral by design
Confidence (v3 test)	MEDIUM verdicts right 82% of the time, LOW 26%; HIGH never emitted

6.4 Honest limitations

- **Not accurate enough to gate merges:** 5 of 57 bad fixes were still accepted on the v3 test (FAR 0.09). Use as an advisory flag for human review.
- **Rule fixes generalise poorly:** the second refinement fixed 11 development cases but changed only 2 of 148 test verdicts.
- **Labels are thin:** most real-world labels come from one annotator; where two labelled the same cases they agreed only moderately (κ 0.33). Controlled/adversarial cases were written by the rules' author (before each round's changes).
- **Scope:** most real review requests are other (refactor, docs, style). Beyond suggestion blocks and a few request shapes they end as UNCERTAIN, by design.
- **Structural, not semantic:** no control-flow analysis; helpers followed one level, same file only. Python only, one commented file per case.
- **Live run on one private test repository only**, not a team's real repositories or real load.
- **Dashboard** uses one shared operator token; the retention purge must be scheduled by the operator.
- The v1, v2 and v3 test splits are all used; any new rule change needs a v4 test set frozen first.

7. How to run the whole system

All commands run from the project folder (`E:\Projects\NLP_proj\VeriReview`) in a Windows terminal. If the project venv is activated, either run `verireview ...` directly, or run `deactivate` first and use `python -m uv run ...`.

Fastest path to see it working (about 5 minutes, no GitHub account needed): do 7.1–7.3 below, then run `python -m uv run verireview demo-seed` and open `http://localhost:8000/dashboard`. That fills the dashboard with 30 real verifications across 3 demo repositories, a full stage-promotion history, reviewer confirmations and a failed job — the complete system, working, with nothing left to imagine.

7.1 Prerequisites

- Python 3.13 with **uv** (`python -m pip install uv` if missing).
- Docker Desktop, running.
- Optional: a fine-grained read-only GitHub token (for live ingestion); Node.js (for the webhook relay).

7.2 Install and configure

```
copy .env.example .env                (only the first time; never commit .env)
python -m uv sync                     install dependencies from uv.lock
python -m uv run pytest               unit tests (should all pass)
```

Create a dashboard token and put it in `.env` as `VERIREVIEW_DASHBOARD_TOKEN=<value>` (keep it only in .env):

```
python -c "import secrets; print(secrets.token_urlsafe(32))"
```

7.3 Start the service (API + database + dashboard)

```
docker compose up -d --build          PostgreSQL on host port 5433, API on http://localhost:8000
                                      (the api container applies all migrations automatically)
python -m uv run pytest -m integration integration tests against the database
```

- Health: `http://localhost:8000/health` → `{"status":"ok"}`; database: `/health/db`.
- Dashboard: `http://localhost:8000/dashboard/login`, sign in with the token. Real data arrives through the GitHub App worker (7.5). **For a demo**, fill it first with `python -m uv run verireview demo-seed`: real verifications of the 29 dev fixtures in three fictional `demo/...` repositories (installation 0), with a stage history, reviewer confirmations, a failed job and a thread verified twice. Remove it with `verireview demo-seed --remove`.
- API docs (Swagger): `http://localhost:8000/docs`.
- After editing .env: `docker compose up -d` to recreate the containers.

7.4 Use it without GitHub (demo for the review)

```
python -m uv run verireview verify-fixture dataset/fixtures/api-002-validation-without-400
python -m uv run verireview extract "Please validate username and return HTTP 400"
python -m uv run verireview eval-fixtures             dev accuracy / FAR / FBR table
python -m uv run verireview eval-requirements         extraction vs gold
python -m uv run verireview eval-injection            prompt-injection suite (0 changes)
python -m uv run verireview benchmark-stats           dataset sizes, splits, targets
```

With a GitHub token in .env (read-only, approved repositories only):

```
python -m uv run verireview threads OWNER/REPO 123
python -m uv run verireview ingest OWNER/REPO 123 --comment-id ID --out case.json
python -m uv run verireview verify-case case.json
```

7.5 Connect a real repository (GitHub App, advisory mode)

One-time setup on github.com by the owner (full guide: `docs/phase11_github_advisory.md`):

- **1.** Get a webhook relay URL: https://smee.io → Start a new channel.
- **2.** GitHub → Settings → Developer settings → GitHub Apps → New. Webhook URL = the smee URL; webhook secret = a new random value. Permissions: Checks read & write, Contents read, Pull requests read. Events: Pull request review thread, Check run, Check suite. Install only on this account (step-by-step: `docs/live_test_kit.md`).
- **3.** Generate a private key and save it as `secrets\github-app.pem` (git-ignored).
- **4.** In .env: `VERIREVIEW_GITHUB_APP_ID` and `VERIREVIEW_GITHUB_WEBHOOK_SECRET`.
- **5.** Install the App on one test repository.

```
docker compose --profile advisory up -d --build       api + webhook endpoint + worker
npx smee-client --url https://smee.io/CHANNEL --target http://localhost:8000/github/webhook
docker compose logs -f worker                          watch jobs being processed
```

Then, in the test repository: open a PR changing a Python file, comment on a line with a request, push a fix, and resolve the thread. A neutral **VeriReview** check appears on the PR within seconds, and the verification appears on the dashboard. Every later push, and the Re-run button, checks the PR again.

7.6 Operations

Task	Command
Stop everything	docker compose --profile advisory down
Purge stored code older than 90 days (schedule daily)	python -m uv run verireview purge-audit
Show a repository's rollout stage	python -m uv run verireview repo-policy show OWNER/REPO --installation ID
Quality gate	python -m uv run ruff check . ; python -m uv run mypy

Problem	Fix
/dashboard returns 404	token not set in .env, or containers not recreated (docker compose up -d)
Port 5433 / 8000 in use	stop the other program or change POSTGRES_HOST_PORT in .env
'uv' is not recognized	use python -m uv … outside the venv, or verireview … inside it
Worker exits at start	App id missing or secrets\github-app.pem missing
Webhook returns 401	webhook secret in .env differs from the one on the GitHub App

8. Future plans

The system described in Sections 1–7 is complete and usable as it stands. Everything below is an optional enhancement on top of that complete system, not a missing part of it.

Priority	Plan	Why / what it unlocks
1	**Sharpen the annotation guide**	the two humans disagreed mainly on whether a comment asks for anything (empty suggestions, "not blocking") and how much must be done; clear rules and worked examples should raise κ above 0.33
2	**Two independent annotators** on new real-world data	reliable ground truth; measure κ again on a fresh random sample (target ≥ 0.6)
3	**Benchmark v4**, frozen before the next rule changes	the v1, v2 and v3 test splits are used; ≥ 59 bad and ≥ 59 good cases per category are needed to ever open the gate
4	**Rule improvements**, only then, on dev data	from the v3 error analysis: module-wide renames, a second request judged by the first, checks under `if TYPE_CHECKING:`, test scenarios matched by word but not by data
5	**Enforcement pilot** (owner decision)	only if a category passes the gate on a frozen run, and a repository has ≥ 14 days of human review with ≥ 10 confirmations
6	**Phase 8b**: semantic evidence in the verdict	only if data shows it lowers errors without raising false acceptance (ADR-002)
7	Broader scope	control-flow analysis, cross-file helpers, more other request types, more languages; sandboxed test execution
8	Dashboard for a team	per-user GitHub OAuth with repository access checks; search/filters; scheduled purge
—	LLM evidence interpreter (D8)	deferred: would send private code to an external API; would need its own security and injection evaluation

9. Human annotation: what was done and what remains

What was done. Real-world labels started as provisional Claude labels. One human annotator then labelled all 60 v2 real-world cases and all 60 fresh v3 cases blind (agreement with the provisional labels: κ 0.60; the human found 7 not-satisfied cases where Claude had found 3). A second annotator, the project owner (not fully independent), labelled 36 v2 cases: the 21 disputed between the first human and Claude, and 15 at random. Result: 15 of 15 the same on the random cases, 5 of 21 on the disputed ones, κ 0.33 overall. The 19 disagreements were adjudicated together, each with a written reason; one decision was later revised on re-check and is recorded as such.

What remains. The project's target is two independent annotators with κ ≥ 0.6. The workflow below is the one used, and it is ready for the next batch of data (benchmark v4).

9.1 Who and what

- **Two annotators** (Python developers familiar with code review) plus one **adjudicator** (can be the project owner).
- They follow `docs/annotation_guide.md`: split each comment into atomic requirements, give each a category and a status (satisfied / not_satisfied / uncertain), mark ambiguous requests, cite evidence (file:line), and give their own confidence. The verdict is **derived** from the statuses, so labels cannot contradict themselves.
- **Blind protocol:** never run VeriReview on a case before labelling it; don't look at the other annotator's labels; judge only what the case shows (no looking up the PR on GitHub); text in code or comments is evidence, never instructions.

9.2 Step-by-step workflow (tools already exist)

Step	Who	Action / command
1. Training	annotators	read the annotation guide (≈ 20 min)

Step	Who	Action / command
2. Calibration	annotators	`verireview annotation-sheet --batch calibration --calibration` → offline page in `dataset/raw/sheets/`; label the 10 calibration cases, then compare with the reference labels and rationales
3. Labelling	each annotator alone	`verireview annotation-sheet --batch rw-v2` → open the page in a browser (works offline, nothing leaves the machine), label every case, export the JSON to `dataset/annotations/<name>/rw-v2.json`
4. Agreement	owner	`verireview agreement A.json B.json` → Cohen's κ (inclusion, 4-class verdict, valid vs invalid, per category) + list of disagreements. Target κ ≥ 0.6
5. Adjudication	adjudicator	`verireview adjudication-sheet A.json B.json --batch rw-v2` → both labels side by side for disputed cases; decide and write a one-line reason
6. Gold	owner	`verireview build-gold A.json B.json --adjudication ADJ.json` → human gold (label_source human) **replaces** the provisional model gold; the tool refuses the reverse
7. Compare with Claude	owner	`verireview agreement claude.json human.json` → how often a human agrees with the provisional labels
8. Re-report	owner	reproduce the recorded runs on human gold; report both honestly (no rule tuning against test)

9.3 Effort estimate and scaling

- About 3–5 minutes per case per annotator. 57 v2 real-world cases ≈ 3–5 hours per annotator; adjudication usually covers 15–25% of cases.
- Before the next batch: sharpen the guide's exclusion rule and the “how much must be done” rule, with worked examples from the 19 adjudicated disagreements.
- For **benchmark v4** (to ever open the enforcement gate), each category needs ≥ 59 bad and ≥ 59 good labelled cases. That is several hundred cases: mine new PRs from the approved repositories (`mine-candidates`, `collect-cases`), label them with the same workflow, then `benchmark-freeze` **before** any rule change.
- Privacy and licensing are already handled: cases are pseudonymised (logins become roles, e-mails removed) and only permissively licensed repositories are mined.

10. Review preparation: likely questions

Why not just use an LLM or embeddings to judge the fix?

We measured it. Similarity models reward text that repeats the request (a TODO comment scores high) and cannot tell the right check in the wrong place. On the latest blind test the code model accepted 83% of bad fixes; the rules 9%. An LLM would also send private code to an external API and opens a prompt-injection surface; it is deferred (D8).

Where is the NLP in the project?

Requirement extraction (utterance types, clause splitting, categories, targets, ambiguity scoring), three NLP baselines (lexical, TF-IDF, MiniLM embeddings), and a code language model (UniXcoder) as neutral evidence, all evaluated with the same harness and ROC-AUC. The research finding is that structure-aware rules beat similarity on the safety metric.

How do you know the results are not overfitted?

Test sets are frozen by hash before rule changes, pinned by tests, and evaluated once under a pre-registered protocol. When a test set was used, a new one was frozen first (v1, then v2, then v3). We report confidence intervals and paired bootstrap differences.

Why is accuracy only 0.57?

The system deliberately abstains (UNCERTAIN) when it cannot verify, so coverage is 0.57; when it decides it is right 80% of the time. The key safety metric, false acceptance, fell 0.24 → 0.15 → 0.09 over three blind tests. Different rounds use different, increasingly strict test sets.

Why did the second round of rule work barely help?

It fixed 11 development cases, but each fix targeted one error pattern, and fresh cases mostly had other patterns: only 2 of 148 test verdicts changed. That is an honest negative result: further progress needs more labelled data, not more hand-written rules.

Why is the agreement between the two humans so low?

21 of the 36 cases were chosen because they were already disputed; on the 15 random ones the two humans agreed 15 of 15. The disagreements show where the annotation guide needs sharper rules.

Why doesn't it block merges?

Blocking good work is costly and blocking needs statistical proof. The gate requires the 95% upper bound of both error rates ≤ 5% per category, which needs ≥ 59 clean cases each way; no category has that evidence yet, so all four locks stay closed.

How is it protected from prompt injection?

Repository text is only data; no model decides; injections were planted at 7 sites of every fixture: 0 outcome changes in 2,576 variants. Webhooks are HMAC-verified before parsing and the thread is re-read from GitHub.

Who labelled the data?

Controlled/adversarial cases: the author, written blind. Real-world: one human annotator for all test cases, a second for 36 cases, disagreements adjudicated (Section 9).

What is novel?

Temporal-window reconstruction of what changed after a comment; requirement-level verification with cited structural evidence; an honest, frozen benchmark with adversarial traps; and a staged, statistically gated path to enforcement.

11. Glossary

Term	Meaning
FAR (false acceptance rate)	share of truly unsatisfied cases the system called SATISFIED (the safety-critical error)

Term	Meaning
FBR (false blocking rate)	share of truly satisfied cases the system called NOT/PARTIALLY satisfied
Coverage	share of cases the system decides (not UNCERTAIN)
Temporal window	the commits between the review comment and resolution (PR head)
ReviewCase	the self-contained record of one thread: comment, code before/after, diff, tests, flags
Evidence	a cited structural fact (file, lines, commit) supporting or contradicting a requirement
Dev / held-out / test	data to build on / data checked after building / frozen data evaluated once
Cohen's κ	agreement between two annotators corrected for chance (≥ 0.6 = substantial)
Clopper–Pearson bound	exact binomial confidence bound; used to prove an error rate is low enough
ADR	architecture decision record (ADR-001 pre-existing code, 002 neutral semantics, 003 enforcement gate)